EDA-Regression-only

Group 8

2026-04-30

Initial data pull/clean

```r
#read the data from the fed website
library(readr)
url <- "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATENSA"
unratensa_data <- read_csv(url)

#convert to time series
unratensa_ts <- ts(unratensa_data$UNRATENSA, start = c(1948, 1), frequency = 12)

#interpolate the NA values
library(forecast)
unratensa_ts_interp <- na.interp(unratensa_ts)

#summary stats
summary(unratensa_ts_interp)
```

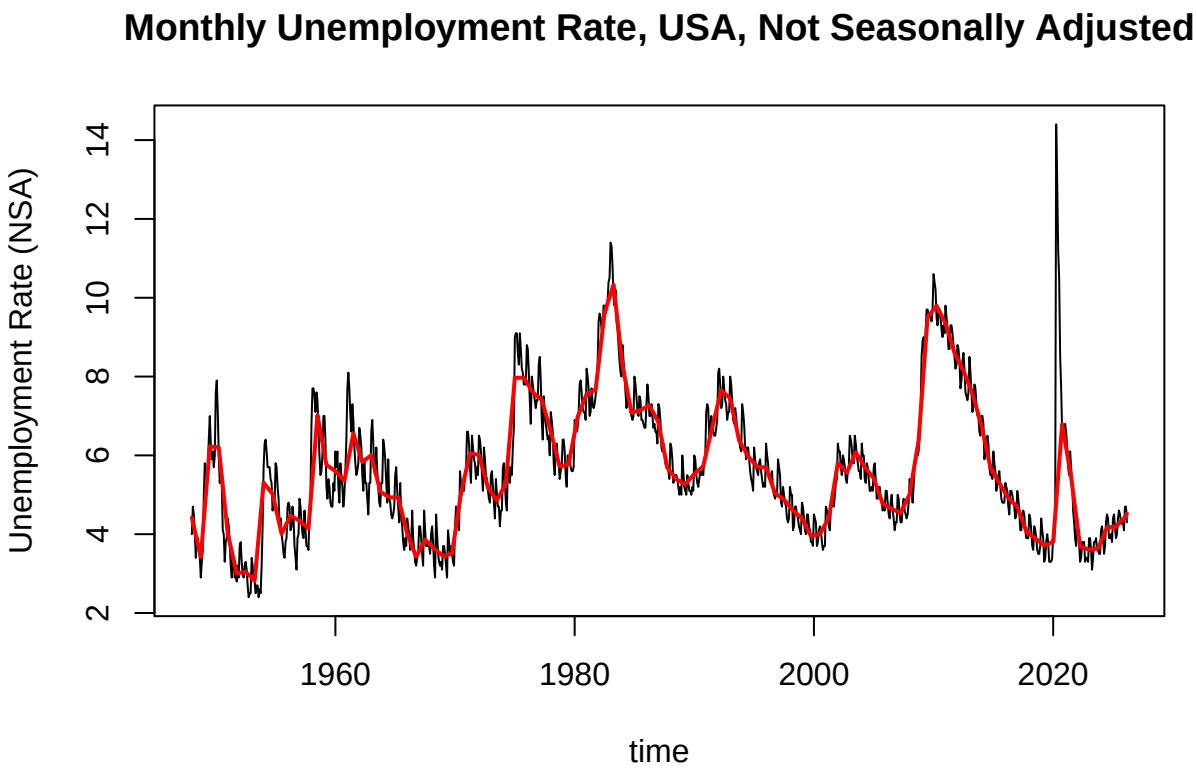
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   2.400   4.400   5.400   5.662   6.700  14.400
```

```r
sd(unratensa_ts_interp)
```

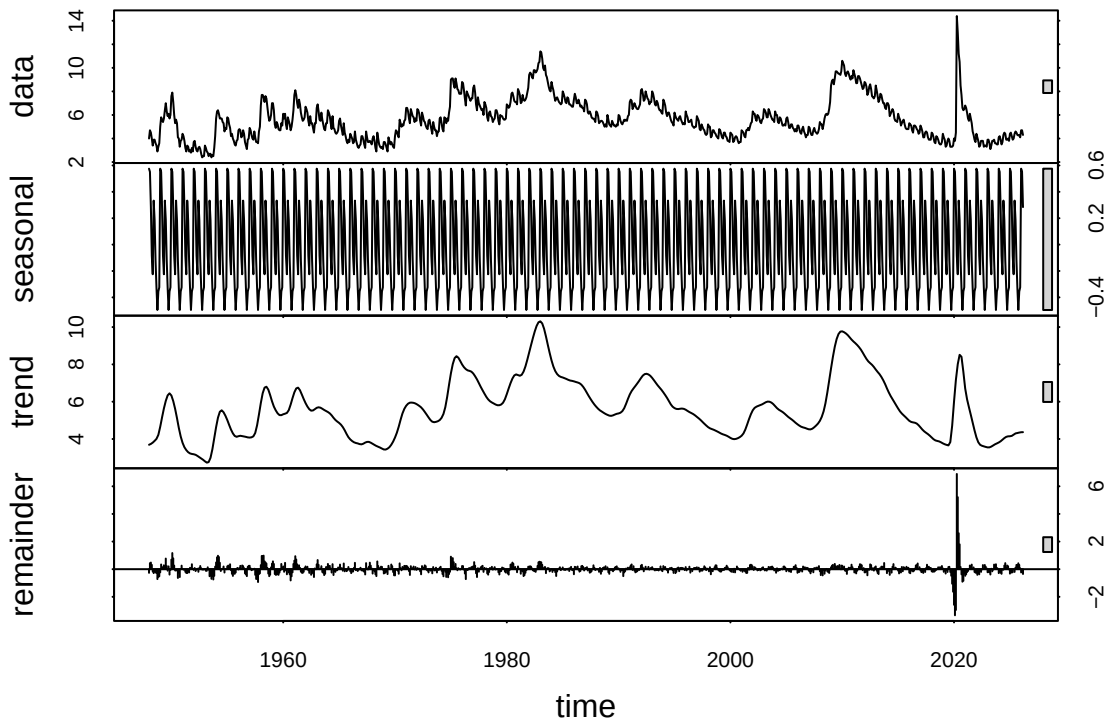
```
## [1] 1.744923
```

smoothing and stl

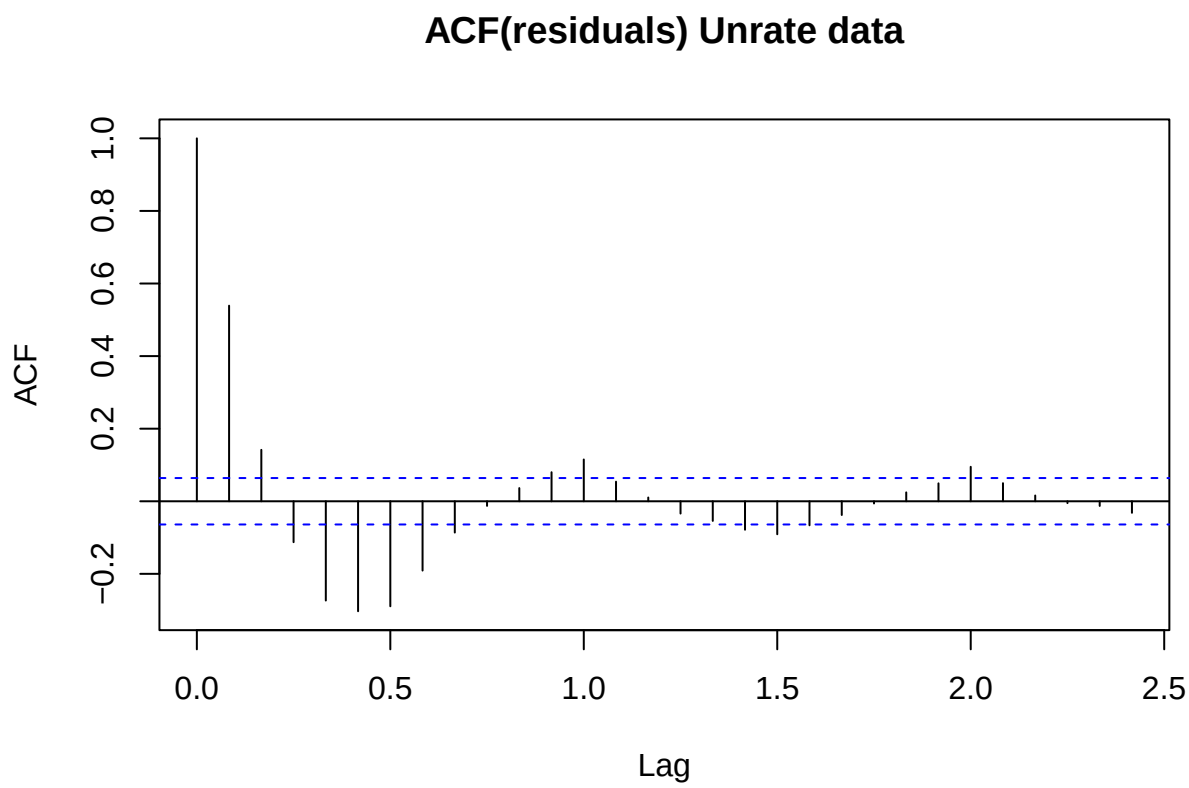
```r
#full data set and smoothing
plot(unratensa_ts_interp, ylab="Unemployment Rate (NSA)", xlab = "time",
     col = "black", main = "Monthly Unemployment Rate, USA, Not Seasonally Adjusted")
lines(lowess(unratensa_ts_interp, f=0.01), lwd=2, col="red")
```




```r
#stl analysis
unratensa_stl <- stl(unratensa_ts_interp, s.window = "periodic")
plot(unratensa_stl)
```




```r
season <- unratensa_stl$time.series[,1]
trend <- unratensa_stl$time.series[,2]
res <- unratensa_stl$time.series[,3]

acf(res, main = "ACF(residuals) Unrate data")
```



Residual AR fit model

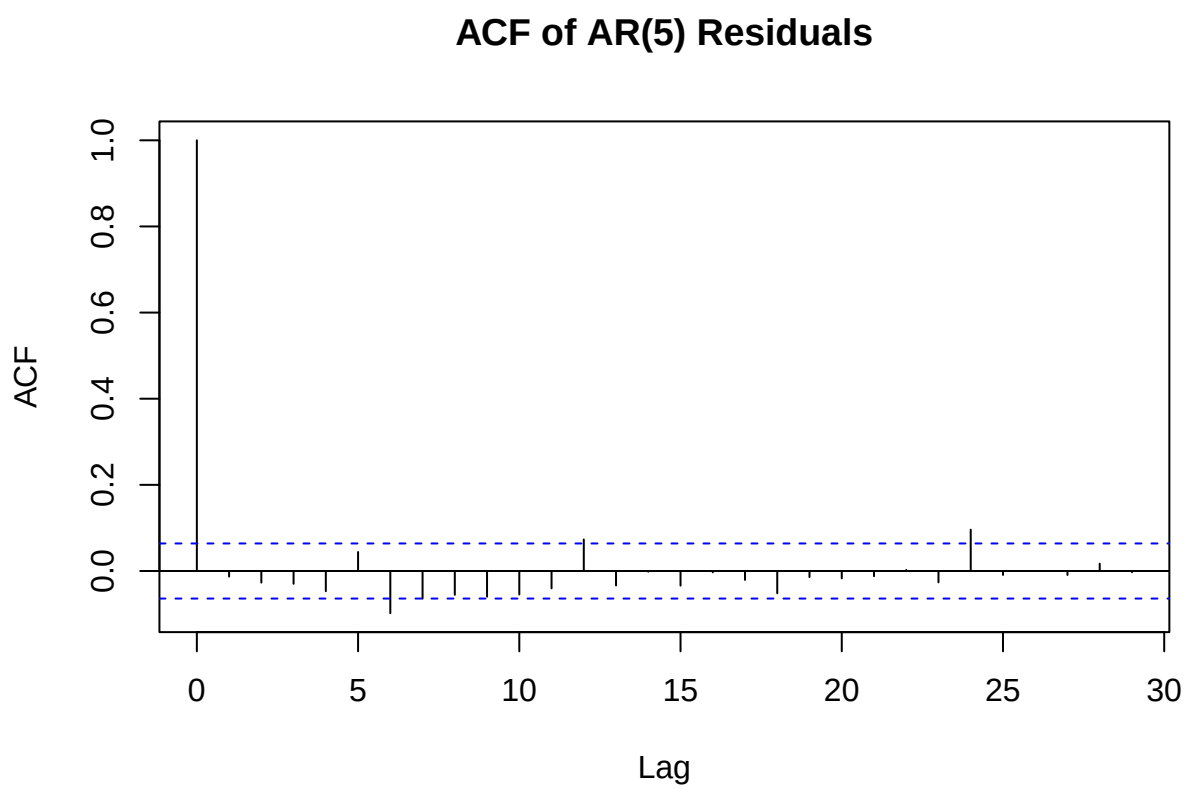
```
#get the order of the AR fit using Yule-Walker
fit_ar <- ar.yw(res, order.max = NULL)
fit_ar
```

```
##
## Call:
## ar.yw.default(x = res, order.max = NULL)
##
## Coefficients:
##       1        2        3        4        5        6        7        8
##  0.5390  -0.1878  -0.0777  -0.1762  -0.0508  -0.1600  -0.0535  -0.0702
##       9       10       11       12       13       14       15       16
## -0.0747  -0.0875  -0.0626   0.0171  -0.1280  -0.0235  -0.0851  -0.0188
##      17       18       19       20       21       22       23       24
## -0.0776  -0.0515  -0.0488  -0.0375  -0.0470  -0.0208  -0.0613   0.0722
##      25
## -0.0894
##
## Order selected 25  sigma^2 estimated as  0.1151
```

```r
#lower order AR
res_clean <- na.omit(as.numeric(res))

# Fit lower-order AR models
ar1 <- arima(res_clean, order = c(1, 0, 0), include.mean = TRUE)
ar2 <- arima(res_clean, order = c(2, 0, 0), include.mean = TRUE)
ar3 <- arima(res_clean, order = c(3, 0, 0), include.mean = TRUE)
ar4 <- arima(res_clean, order = c(4, 0, 0), include.mean = TRUE)
ar5 <- arima(res_clean, order = c(5, 0, 0), include.mean = TRUE)

# Compare lower-order models using AIC
AIC(ar1, ar2, ar3, ar4, ar5)
```

```
##     df      AIC
## ar1  3 779.7643
## ar2  4 739.1523
## ar3  5 723.6950
## ar4  6 698.1354
## ar5  7 692.3861
```

```r
# Check selected AR(5) residuals
acf(residuals(ar5), main = "ACF of AR(5) Residuals")
```



Trend model

```r
time <- 1:length(trend)
dat.trend <- data.frame(trend = trend, time = time)

# polynomial fits
fit_poly3 <- lm(trend ~ time + I(time^2) + I(time^3), data = dat.trend)

fit_poly5 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5), data = dat.trend)

fit_poly8 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5) + I(time^6) +
                  I(time^7) + I(time^8), data = dat.trend)

# summaries
summary(fit_poly3)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.3700 -1.0547 -0.2808  0.8407  3.9877
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.971e+00  1.878e-01  21.149  < 2e-16 ***
## time         7.943e-03  1.729e-03   4.594 4.94e-06 ***
## I(time^2)   -3.613e-06  4.273e-06  -0.846    0.398
## I(time^3)   -4.726e-09  2.988e-09  -1.581    0.114
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.433 on 935 degrees of freedom
## Multiple R-squared:  0.1989, Adjusted R-squared:  0.1963
## F-statistic: 77.37 on 3 and 935 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly5)
```

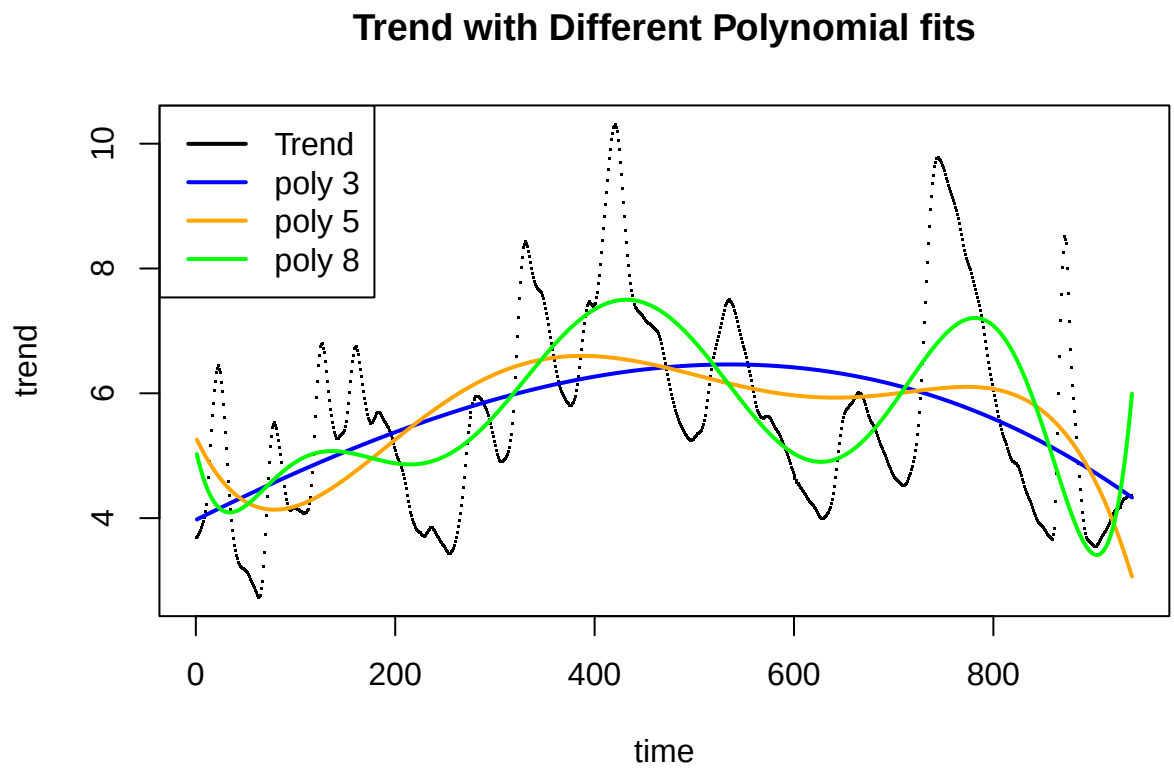
```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.4769 -1.0720 -0.1943  0.8512  3.7451
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)  5.292e+00  2.727e-01  19.408  < 2e-16 ***
## time        -3.385e-02  5.851e-03  -5.786 9.85e-09 ***
## I(time^2)    3.066e-04  3.852e-05   7.960 4.99e-15 ***
## I(time^3)   -8.833e-07  1.038e-07  -8.507  < 2e-16 ***
## I(time^4)    1.051e-09  1.218e-10   8.629  < 2e-16 ***
## I(time^5)   -4.468e-13  5.155e-14  -8.667  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.38 on 933 degrees of freedom
## Multiple R-squared:  0.2586, Adjusted R-squared:  0.2546
## F-statistic: 65.08 on 5 and 933 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly8)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7865 -0.9008 -0.0317  0.6954  4.1089
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.097e+00  3.534e-01  14.425  < 2e-16 ***
## time        -7.246e-02  1.728e-02  -4.194 3.01e-05 ***
## I(time^2)    1.725e-03  2.731e-04   6.317 4.12e-10 ***
## I(time^3)   -1.594e-05  1.960e-06  -8.132 1.34e-15 ***
## I(time^4)    7.314e-08  7.463e-09   9.800  < 2e-16 ***
## I(time^5)   -1.805e-10  1.608e-11 -11.231  < 2e-16 ***
## I(time^6)    2.437e-13  1.966e-14  12.399  < 2e-16 ***
## I(time^7)   -1.694e-16  1.271e-17 -13.323  < 2e-16 ***
## I(time^8)    4.735e-20  3.374e-21  14.033  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.178 on 930 degrees of freedom
## Multiple R-squared:  0.4616, Adjusted R-squared:  0.457
## F-statistic: 99.68 on 8 and 930 DF,  p-value: < 2.2e-16
```

```r
# compare polynomial degrees
plot(time, trend, pch=".", main="Trend with Different Polynomial fits")

lines(time, fitted(fit_poly3), col="blue", lwd=2)
lines(time, fitted(fit_poly5), col="orange", lwd=2)
lines(time, fitted(fit_poly8), col="green", lwd=2)

legend("topleft",
       legend=c("Trend", "poly 3", "poly 5", "poly 8"),
       col=c("black", "blue", "orange", "green"),
       lwd=2)
```




```r
# AIC/BIC selection criteria
AIC(fit_poly3, fit_poly5, fit_poly8)
```

```
##           df      AIC
## fit_poly3  5 3345.936
## fit_poly5  7 3277.219
## fit_poly8 10 2982.724
```

```r
BIC(fit_poly3, fit_poly5, fit_poly8)
```

```
##           df      BIC
## fit_poly3  5 3370.161
## fit_poly5  7 3311.132
## fit_poly8 10 3031.172
```

Seasonal model

```r
time <- 1:length(season)

# month index 1-12 repeating
month <- cycle(unratensa_ts_interp)
```

```r
dat.season <- data.frame(
  season = as.numeric(season),
  time = time,
  month = as.numeric(month)
)

# trig fit, 1 harmonic
fit_season1 <- lm(season ~
                    cos(2*pi*month/12) + sin(2*pi*month/12),
                  data = dat.season)

# trig fit, 3 harmonics
fit_season2 <- lm(season ~
                    cos(2*pi*month/12) + sin(2*pi*month/12) +
                    cos(4*pi*month/12) + sin(4*pi*month/12) +
                    cos(6*pi*month/12) + sin(6*pi*month/12),
                  data = dat.season)

# check model summaries
summary(fit_season1)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * month/12) + sin(2 * pi * month/12),
##     data = dat.season)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.41997 -0.20964 -0.01918  0.29451  0.46633
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             0.0008124  0.0092394   0.088 0.929949
## cos(2 * pi * month/12) -0.0495083  0.0130733  -3.787 0.000162 ***
## sin(2 * pi * month/12)  0.3026840  0.0130595  23.177  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2831 on 936 degrees of freedom
## Multiple R-squared:  0.3706, Adjusted R-squared:  0.3693
## F-statistic: 275.6 on 2 and 936 DF,  p-value: < 2.2e-16
```

```r
summary(fit_season2)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * month/12) + sin(2 * pi * month/12) +
##     cos(4 * pi * month/12) + sin(4 * pi * month/12) + cos(6 *
##     pi * month/12) + sin(6 * pi * month/12), data = dat.season)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
```

```
## -0.157399 -0.081057  0.004208  0.094241  0.170761
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             6.604e-05  3.211e-03   0.021 0.983597
## cos(2 * pi * month/12) -5.051e-02  4.544e-03 -11.117  < 2e-16 ***
## sin(2 * pi * month/12)  3.016e-01  4.539e-03  66.453  < 2e-16 ***
## cos(4 * pi * month/12)  3.671e-02  4.541e-03   8.084 1.94e-15 ***
## sin(4 * pi * month/12)  3.400e-01  4.541e-03  74.875  < 2e-16 ***
## cos(6 * pi * month/12) -1.525e-01  4.544e-03 -33.559  < 2e-16 ***
## sin(6 * pi * month/12) -1.513e-02  4.539e-03  -3.333 0.000894 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0984 on 932 degrees of freedom
## Multiple R-squared:  0.9243, Adjusted R-squared:  0.9238
## F-statistic:  1897 on 6 and 932 DF,  p-value: < 2.2e-16
```

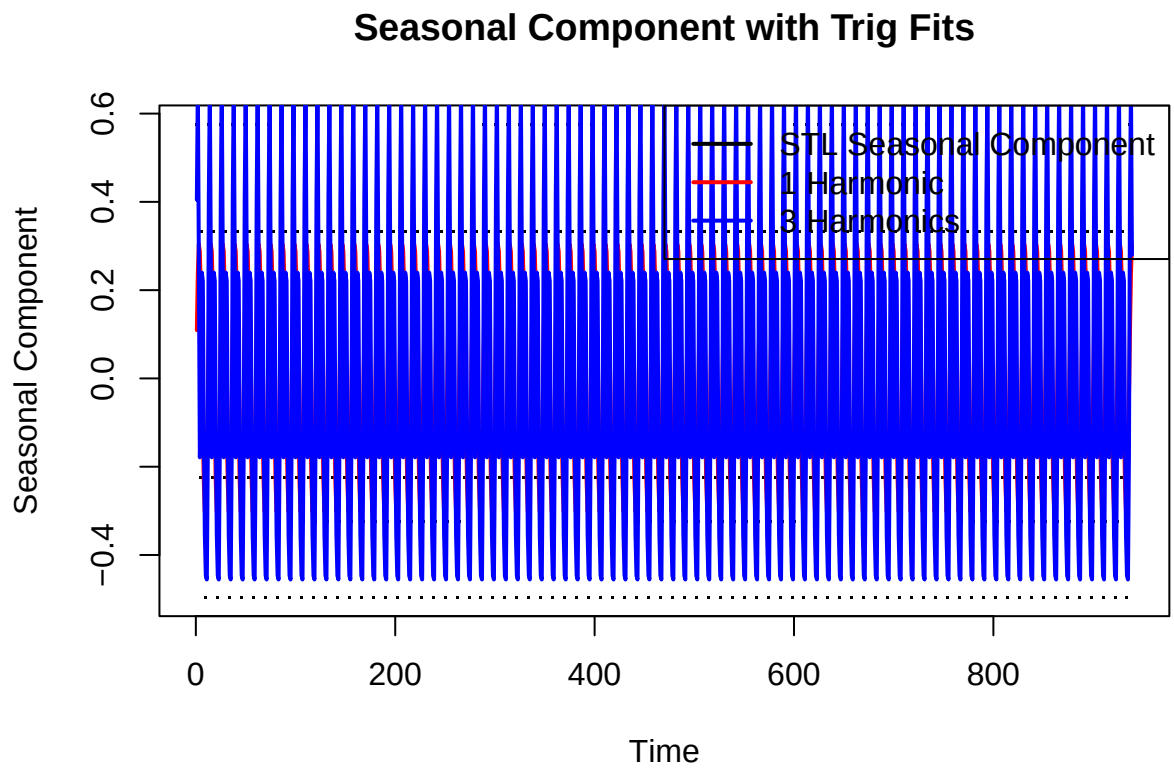
```r
# AIC/BIC comparison
AIC(fit_season1, fit_season2)
```

```
##             df        AIC
## fit_season1  4   299.9439
## fit_season2  8 -1680.7950
```

```r
BIC(fit_season1, fit_season2)
```

```
##             df        BIC
## fit_season1  4   319.3231
## fit_season2  8 -1642.0365
```

```r
# fitted seasonal values
season_fit1 <- fitted(fit_season1)
season_fit2 <- fitted(fit_season2)

# full plot
plot(time, season, pch=".", col="black",
     main="Seasonal Component with Trig Fits",
     xlab="Time", ylab="Seasonal Component")

lines(time, season_fit1, col="red", lwd=2)
lines(time, season_fit2, col="blue", lwd=2)

legend("topright",
       legend=c("STL Seasonal Component", "1 Harmonic", "3 Harmonics"),
       col=c("black", "red", "blue"),
       lwd=2)
```



Combined Models

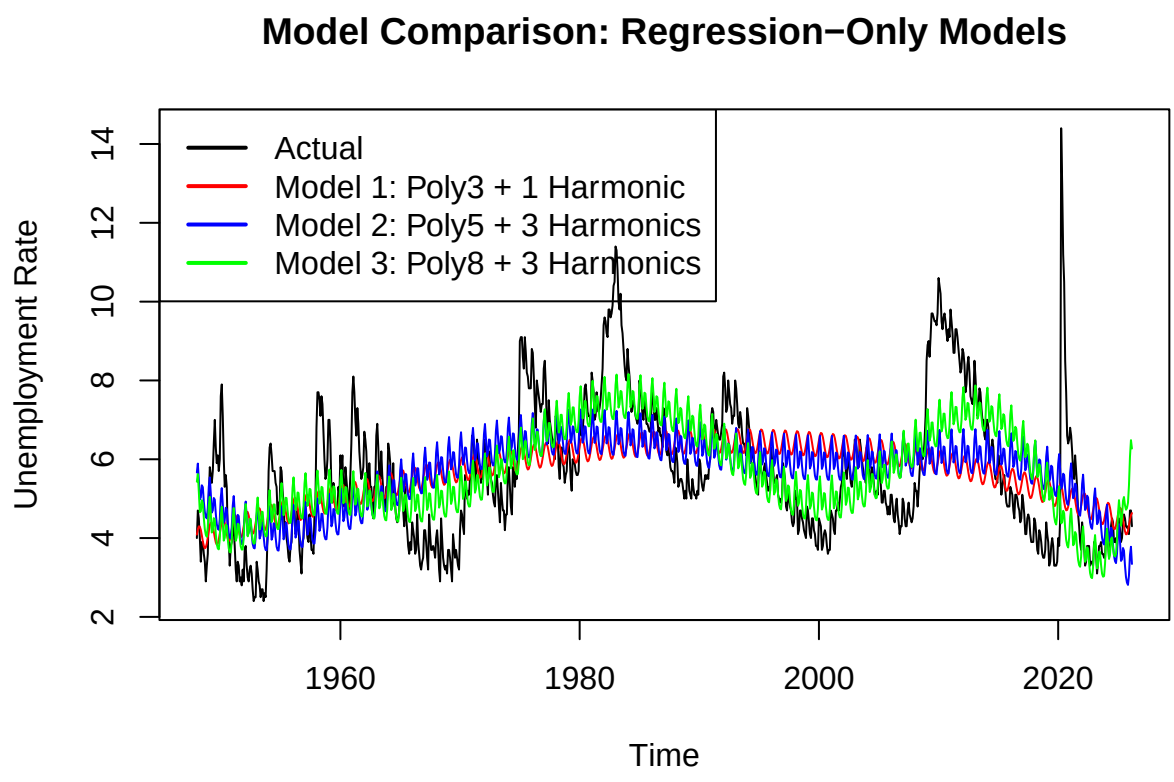
```
#trend comp
tr_poly3 <- fitted(fit_poly3)
tr_poly5 <- fitted(fit_poly5)
tr_poly8 <- fitted(fit_poly8)

#seasonal comp, monthly cycle, cycle() fixed to a 1-12 cycle
month <- cycle(unratensa_ts_interp)

# rebuild seasonal fits
coef_s1 <- coef(fit_season1)
coef_s2 <- coef(fit_season2)

ss1 <- coef_s1[1] +
       coef_s1[2]*cos(2*pi*month/12) +
       coef_s1[3]*sin(2*pi*month/12)

ss2 <- coef_s2[1] +
       coef_s2[2]*cos(2*pi*month/12) +
       coef_s2[3]*sin(2*pi*month/12) +
       coef_s2[4]*cos(4*pi*month/12) +
       coef_s2[5]*sin(4*pi*month/12) +
       coef_s2[6]*cos(6*pi*month/12) +
```

```r
       coef_s2[7]*sin(6*pi*month/12)

#combined regression-only models
yhat1 <- tr_poly3 + ss1                 # simple
yhat2 <- tr_poly5 + ss2                 # balanced
yhat3 <- tr_poly8 + ss2                 # flexible / overfit comparison

# convert fitted values to time series
yhat1_ts <- ts(yhat1, start=start(unratensa_ts_interp), frequency=12)
yhat2_ts <- ts(yhat2, start=start(unratensa_ts_interp), frequency=12)
yhat3_ts <- ts(yhat3, start=start(unratensa_ts_interp), frequency=12)

# final residuals for regression-only models
final_resid1 <- unratensa_ts_interp - yhat1_ts
final_resid2 <- unratensa_ts_interp - yhat2_ts
final_resid3 <- unratensa_ts_interp - yhat3_ts

# Function to add AR(5) residual errors
add_ar5_errors <- function(yhat_ts, actual_ts) {
  resid <- actual_ts - yhat_ts

  ar5_fit <- arima(as.numeric(resid),
                   order = c(5, 0, 0),
                   include.mean = TRUE)

  ar5_resid <- residuals(ar5_fit)
  ar5_component <- as.numeric(resid) - ar5_resid

  yhat_ar5 <- as.numeric(yhat_ts) + ar5_component

  return(ts(yhat_ar5,
            start = start(actual_ts),
            frequency = frequency(actual_ts)))
}

# AR(5)-adjusted models
yhat1_ar5_ts <- add_ar5_errors(yhat1_ts, unratensa_ts_interp)
yhat2_ar5_ts <- add_ar5_errors(yhat2_ts, unratensa_ts_interp)
yhat3_ar5_ts <- add_ar5_errors(yhat3_ts, unratensa_ts_interp)

# final residuals for AR(5)-adjusted models
final_resid1_ar5 <- unratensa_ts_interp - yhat1_ar5_ts
final_resid2_ar5 <- unratensa_ts_interp - yhat2_ar5_ts
final_resid3_ar5 <- unratensa_ts_interp - yhat3_ar5_ts
```

Model comparison plots

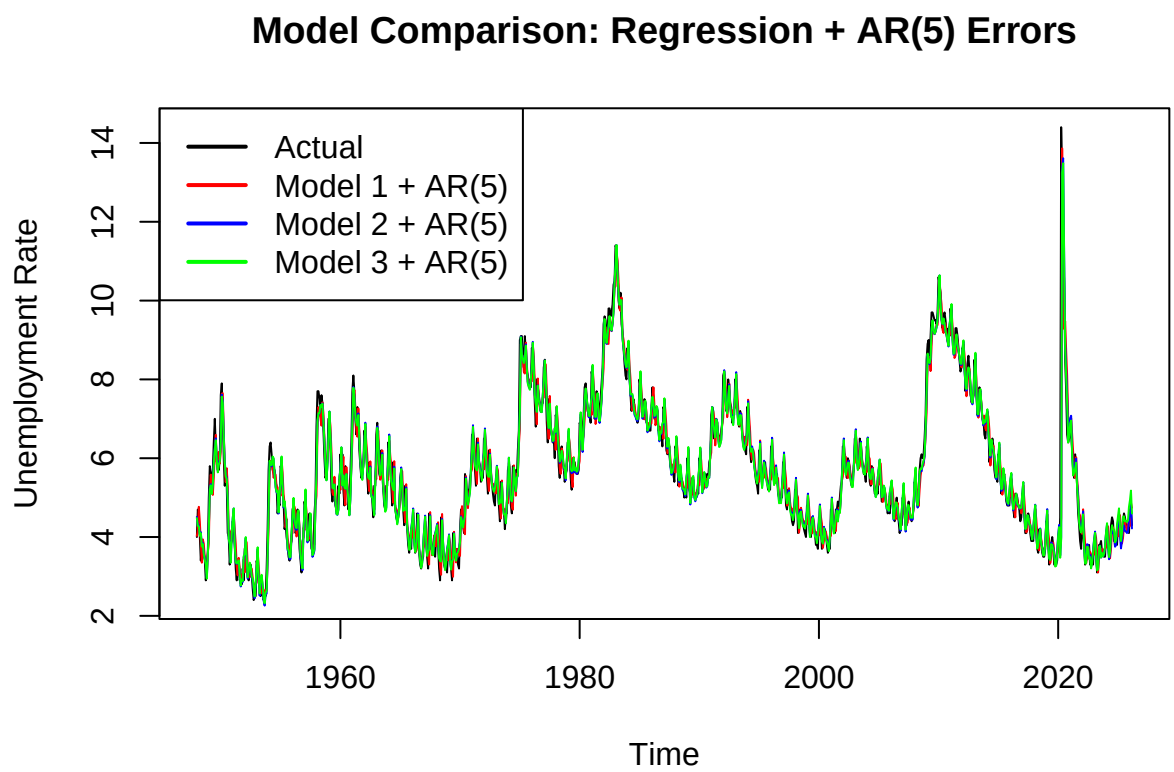
```r
#Regression-only models
plot(unratensa_ts_interp, col="black",
     main="Model Comparison: Regression-Only Models",
     ylab="Unemployment Rate")
```

```r
lines(yhat1_ts, col="red")
lines(yhat2_ts, col="blue")
lines(yhat3_ts, col="green")

legend("topleft",
       legend=c("Actual",
                "Model 1: Poly3 + 1 Harmonic",
                "Model 2: Poly5 + 3 Harmonics",
                "Model 3: Poly8 + 3 Harmonics"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```



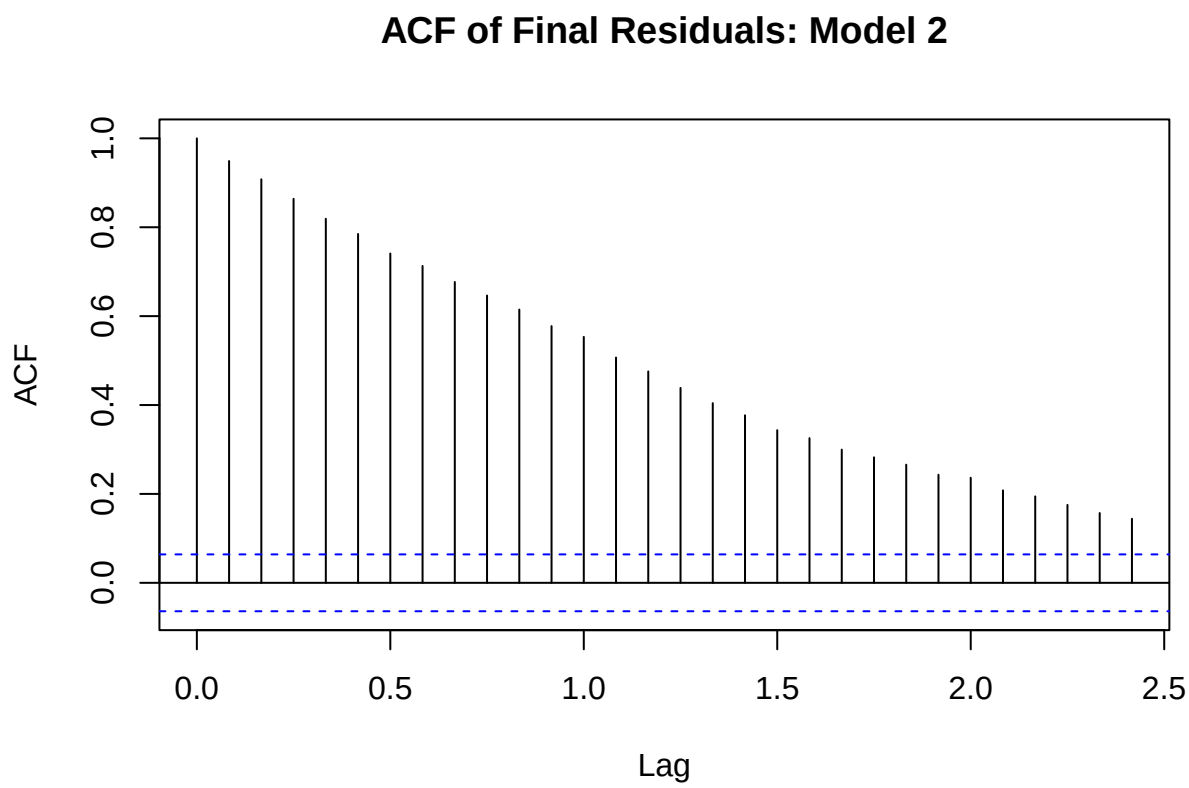

```r
#AR(5)-adjusted models
plot(unratensa_ts_interp, col="black",
     main="Model Comparison: Regression + AR(5) Errors",
     ylab="Unemployment Rate")

lines(yhat1_ar5_ts, col="red")
lines(yhat2_ar5_ts, col="blue")
lines(yhat3_ar5_ts, col="green")

legend("topleft",
       legend=c("Actual",
                "Model 1 + AR(5)",
                "Model 2 + AR(5)",
```

```
                "Model 3 + AR(5)"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```

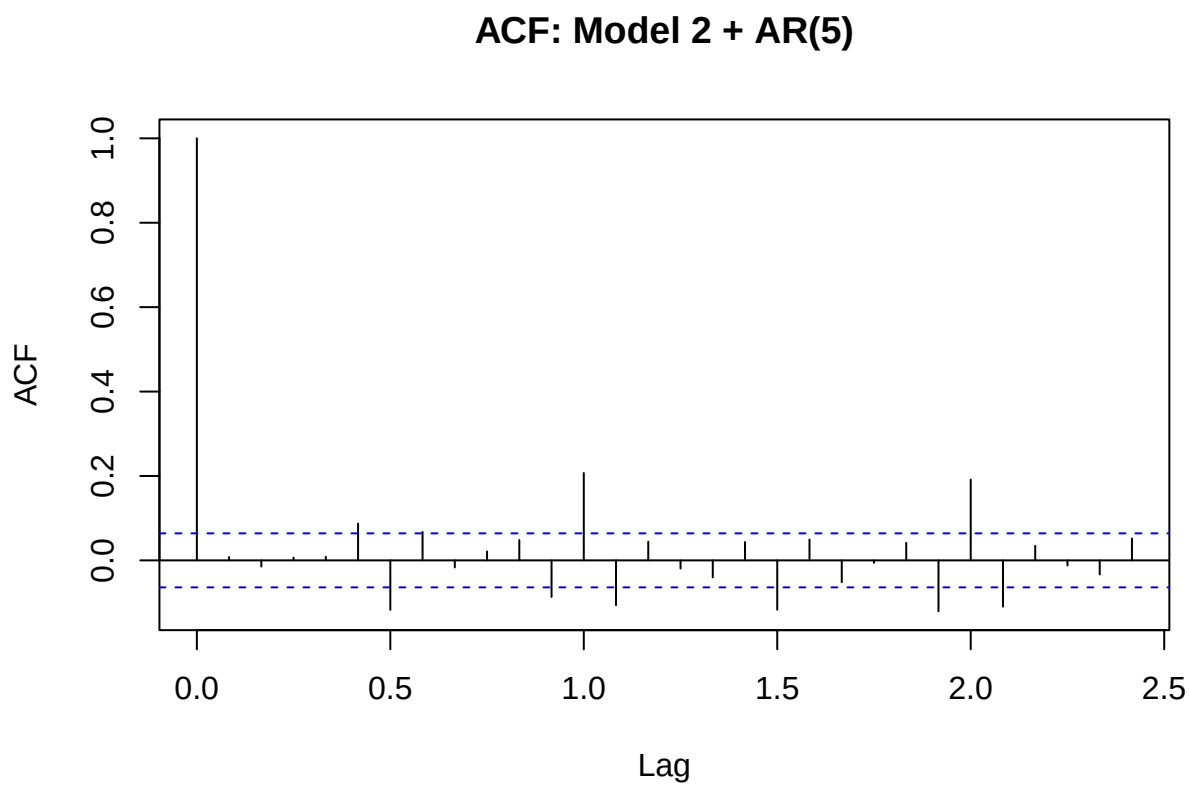


Residual diagnostics

```
# ACF comparisons for selected balanced model
acf(final_resid2, main="ACF of Final Residuals: Model 2")
```




```
acf(final_resid2_ar5, main="ACF: Model 2 + AR(5)")
```




```r
# Ljung-Box residual checks
Box.test(final_resid1, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid1
## X-squared = 6316.1, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid2, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid2
## X-squared = 6350.2, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid3, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid3
## X-squared = 5243.4, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid1_ar5, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid1_ar5
## X-squared = 179.8, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid2_ar5, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid2_ar5
## X-squared = 75.741, df = 12, p-value = 2.661e-11
```

```r
Box.test(final_resid3_ar5, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid3_ar5
## X-squared = 76.754, df = 12, p-value = 1.71e-11
```